

ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 5 - Due date 02/17/26

Sadie Kasten

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., “LuanaLima_TSA_A05_Sp26.Rmd”). Then change “Student Name” on line 3 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Canvas.

R packages needed for this assignment: “readxl”, “ggplot2”, “forecast”, “tseries”, and “Kendall”. Install these packages, if you haven’t done yet. Do not forget to load them before running your script, since they are NOT default packages.\

```
#Load/install required package here
```

```
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
```

```
##   method      from
```

```
##   as.zoo.data.frame zoo
```

```
library(tseries)
```

```
library(ggplot2)
```

```
library(Kendall)
```

```
library(lubridate)
```

```
##
```

```
## Attaching package: 'lubridate'
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##   date, intersect, setdiff, union
```

```
library(tidyverse) #load this package so you can clean the data frame using pipes
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
## v dplyr 1.1.4 v stringr 1.5.1  
## v forcats 1.0.0 v tibble 3.2.1  
## v purrr 1.0.2 v tidyr 1.3.1  
## v readr 2.1.5
```

```
## -- Conflicts ----- tidyverse_conflicts() --  
## x dplyr::filter() masks stats::filter()  
## x dplyr::lag() masks stats::lag()  
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(readxl)
```

Consider the same data you used for A04 from the spreadsheet “Table_10.1_Renewable_Energy_Production_and_Consumption”. The data comes from the US Energy Information Administration and corresponds to the December 2025 Monthly Energy Review.

```
rm(list = ls())
```

```
#Importing data set - using readxl package  
energy_data <- read_excel(  
  path = ./Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx,  
  skip = 12,  
  sheet = "Monthly Data",  
  col_names = FALSE  
)
```

```
## New names:  
## * ' ' -> '...1'  
## * ' ' -> '...2'  
## * ' ' -> '...3'  
## * ' ' -> '...4'  
## * ' ' -> '...5'  
## * ' ' -> '...6'  
## * ' ' -> '...7'  
## * ' ' -> '...8'  
## * ' ' -> '...9'  
## * ' ' -> '...10'  
## * ' ' -> '...11'  
## * ' ' -> '...12'  
## * ' ' -> '...13'  
## * ' ' -> '...14'
```

```
#Now let's extract the column names from row 11 only  
read_col_names <- read_excel(  
  path = ./Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx,  
  skip = 10,  
  n_max = 1,  
  sheet = "Monthly Data",  
  col_names = FALSE  
)
```

```
## New names:
## * ' ' -> '...1'
## * ' ' -> '...2'
## * ' ' -> '...3'
## * ' ' -> '...4'
## * ' ' -> '...5'
## * ' ' -> '...6'
## * ' ' -> '...7'
## * ' ' -> '...8'
## * ' ' -> '...9'
## * ' ' -> '...10'
## * ' ' -> '...11'
## * ' ' -> '...12'
## * ' ' -> '...13'
## * ' ' -> '...14'
```

```
colnames(energy_data) <- read_col_names
nobs <- nrow(energy_data)

nobs=nrow(energy_data)
nvar=ncol(energy_data)

head(energy_data)
```

```
## # A tibble: 6 x 14
##   Month      'Wood Energy Production' 'Biofuels Production'
##   <dtm>                                <dbl> <chr>
## 1 1973-01-01 00:00:00                130. Not Available
## 2 1973-02-01 00:00:00                117. Not Available
## 3 1973-03-01 00:00:00                130. Not Available
## 4 1973-04-01 00:00:00                125. Not Available
## 5 1973-05-01 00:00:00                130. Not Available
## 6 1973-06-01 00:00:00                125. Not Available
## # i 11 more variables: 'Total Biomass Energy Production' <dbl>,
## #   'Total Renewable Energy Production' <dbl>,
## #   'Hydroelectric Power Consumption' <dbl>,
## #   'Geothermal Energy Consumption' <dbl>, 'Solar Energy Consumption' <chr>,
## #   'Wind Energy Consumption' <chr>, 'Wood Energy Consumption' <dbl>,
## #   'Waste Energy Consumption' <dbl>, 'Biofuels Consumption' <chr>,
## #   'Total Biomass Energy Consumption' <dbl>, ...
```

Handling Missing Data

Q1

Using the original dataset, create a new data frame that includes only the following variables: **Date**, **Solar Energy Consumption** and **Wind Energy Consumption**. Check the class of columns, you will see that they are stored as characters instead of numbers. Because solar generation begins later in the sample, the early observations are recorded as “Not Available”. Convert the data to numeric, the “Not Available” will become NAs.

You may either filter out the “Not Available” rows and then convert the column to numeric or convert first and then remove missing values using `drop_na()` (or `na.omit()`). If you are comfortable using pipes for data wrangling, please do so.

Important: Note that we dropping the missing observations instead of interpolating is because they only happen in the beginning of the series!

```
new_df <- energy_data[,c("Month", "Solar Energy Consumption", "Wind Energy Consumption")]  
class(new_df$`Wind Energy Consumption`)
```

```
## [1] "character"
```

```
new_df$`Wind Energy Consumption` <- as.numeric(new_df$`Wind Energy Consumption`)
```

```
## Warning: NAs introduced by coercion
```

```
new_df$`Solar Energy Consumption` <- as.numeric(new_df$`Solar Energy Consumption`)
```

```
## Warning: NAs introduced by coercion
```

```
new_df$Month <- as.Date(new_df$Month) #needed to do this to do the plotting thing below with scale_x...  
class(new_df$Month)
```

```
## [1] "Date"
```

```
#not using pipes, first converted columns to numeric and now removing na lines  
#now starts in January 1984  
filtered_df <- drop_na(new_df)  
head(filtered_df)
```

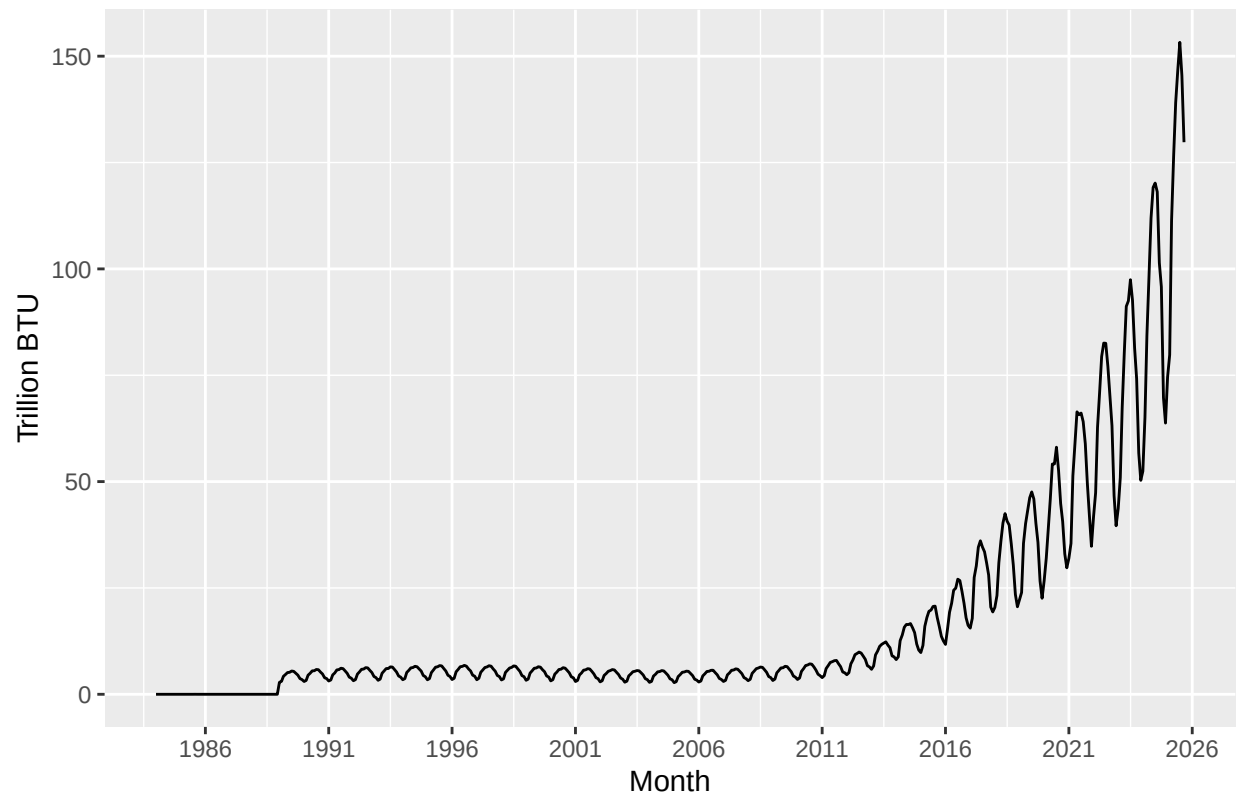
```
## # A tibble: 6 x 3  
##   Month      'Solar Energy Consumption' 'Wind Energy Consumption'  
##   <date>                <dbl>                <dbl>  
## 1 1984-01-01              0                  0  
## 2 1984-02-01              0                0.001  
## 3 1984-03-01            0.001                0.001  
## 4 1984-04-01            0.001                0.002  
## 5 1984-05-01            0.002                0.003  
## 6 1984-06-01            0.003                0.002
```

Q2

Plot the Solar and Wind energy consumption over time using ggplot. Plot each series on a separate graph. No need to add legend. Add informative names to the y axis using `ylab()`. Explore the function `scale_x_date()` on ggplot and see if you can change the x axis to improve your plot. Hint: use `scale_x_date(date_breaks = "5 years", date_labels = "%Y")`

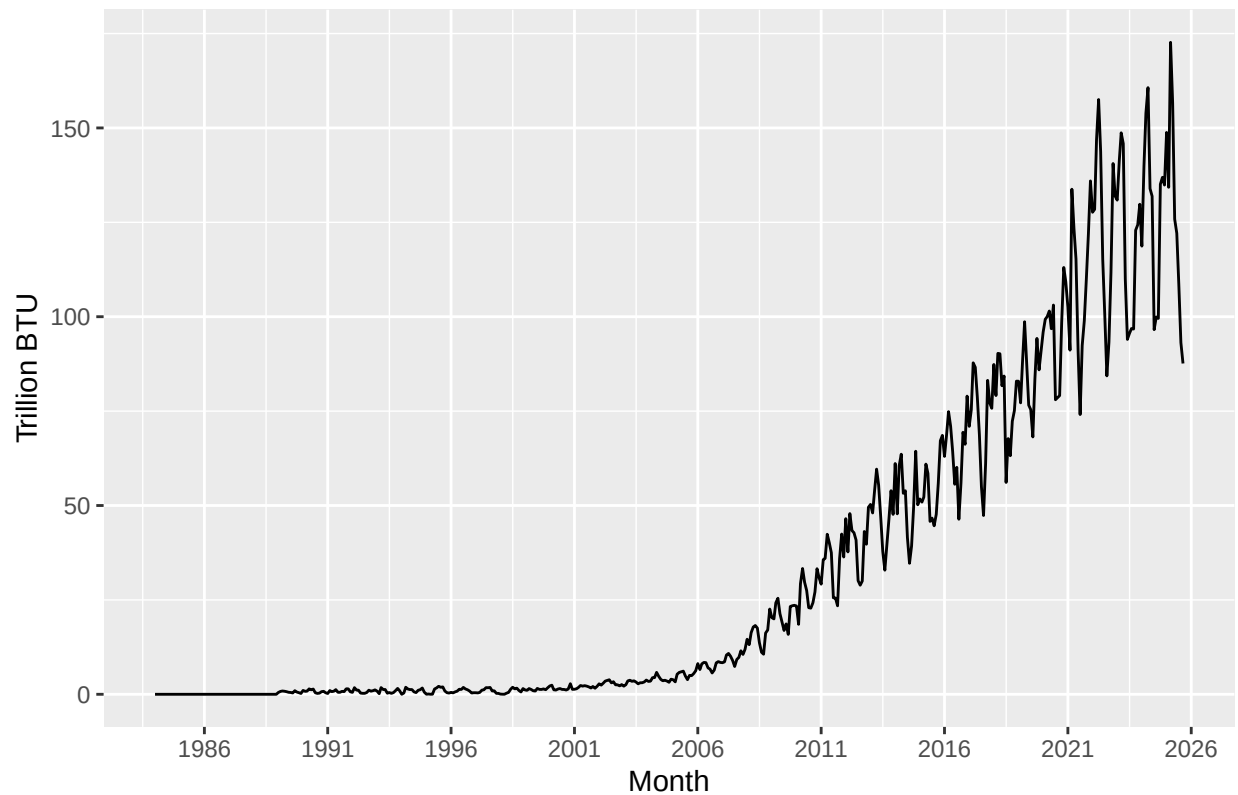
```
#solar energy plot  
ggplot(filtered_df, aes(`Month`, `Solar Energy Consumption`))+  
  geom_line()+  
  labs(title = "US Solar Energy Consumption over Time")+  
  ylab("Trillion BTU")+  
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```

US Solar Energy Consumption over Time



```
#wind energy plot  
ggplot(filtered_df, aes(`Month`, `Wind Energy Consumption`))+  
  geom_line()+  
  labs(title = "US Wind Energy Consumption over Time")+  
  ylab("Trillion BTU")+  
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```

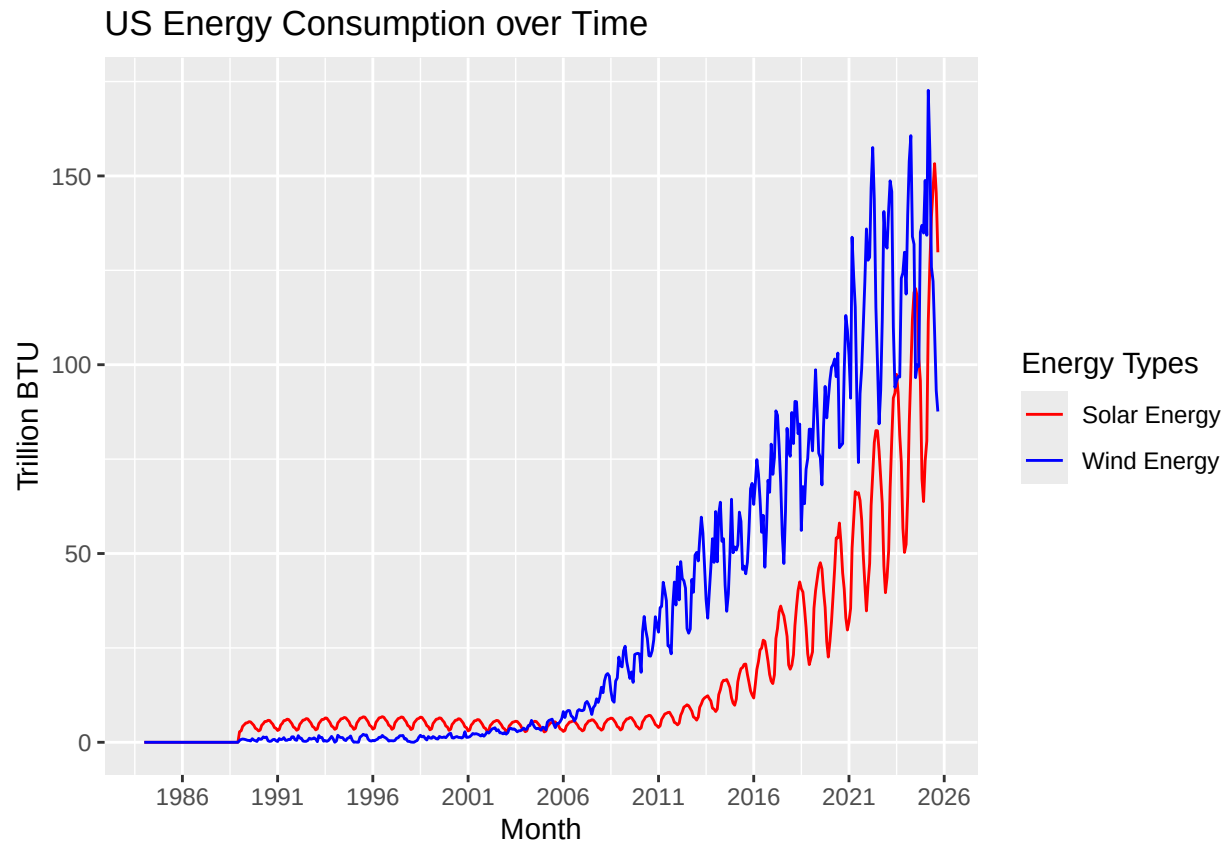
US Wind Energy Consumption over Time



Q3

Now plot both series in the same graph, also using `ggplot()`. Use function `scale_color_manual()` to manually add a legend to `ggplot`. Make the solar energy consumption red and wind energy consumption blue. Add informative name to the y axis using `ylab("Energy Consumption")`. And use function `scale_x_date()` to set x axis breaks every 5 years.

```
ggplot(filtered_df)+
  geom_line(aes(`Month`, `Solar Energy Consumption`, color = 'Solar Energy')) +
  geom_line(aes(`Month`, `Wind Energy Consumption`, color = 'Wind Energy')) +
  scale_color_manual(name = 'Energy Types', values=c('Solar Energy'="red",'Wind Energy'="blue"))+
  labs(title = "US Energy Consumption over Time")+
  ylab("Trillion BTU")+
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")
```



Decomposing the time series

The stats package has a function called `decompose()`. This function only take time series object. As the name says the decompose function will decompose your time series into three components: trend, seasonal and random. This is similar to what we did in the previous script, but in a more automated way. The random component is the time series without seasonal and trend component.

Additional info on `decompose()`.

- 1) You have two options: alternative and multiplicative. Multiplicative models exhibit a change in frequency over time.
- 2) The trend is not a straight line because it uses a moving average method to detect trend.
- 3) The seasonal component of the time series is found by subtracting the trend component from the original data then grouping the results by month and averaging them.
- 4) The random component, also referred to as the noise component, is composed of all the leftover signal which is not explained by the combination of the trend and seasonal component.

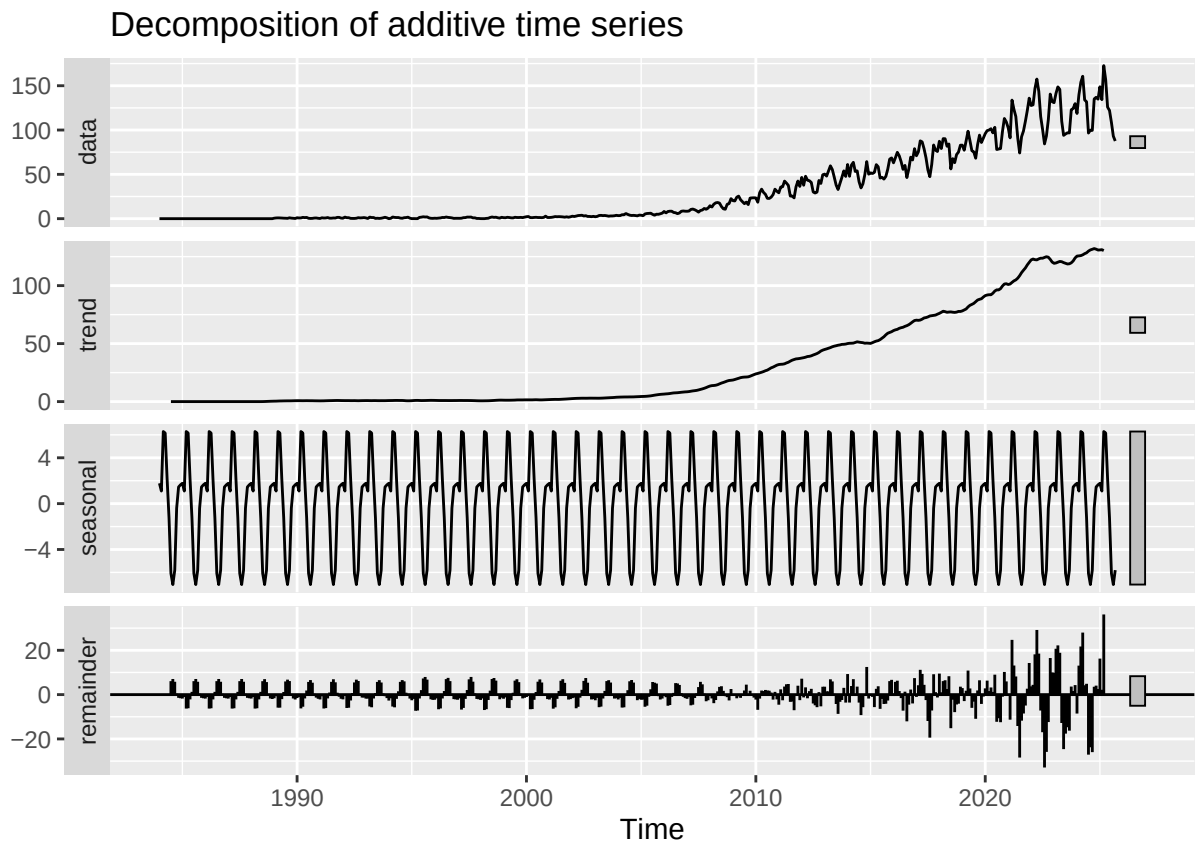
Q4

Transform wind and solar series into a time series object and apply the decompose function on them using the additive option, i.e., `decompose(ts_data, type = "additive")`. What can you say about the trend component? What about the random component? Does the random component look random? Or does it appear to still have some seasonality on it?

```
#creating a time series
wind_ts <- ts(filtered_df$`Wind Energy Consumption`, frequency = 12, start = c(1984,1))

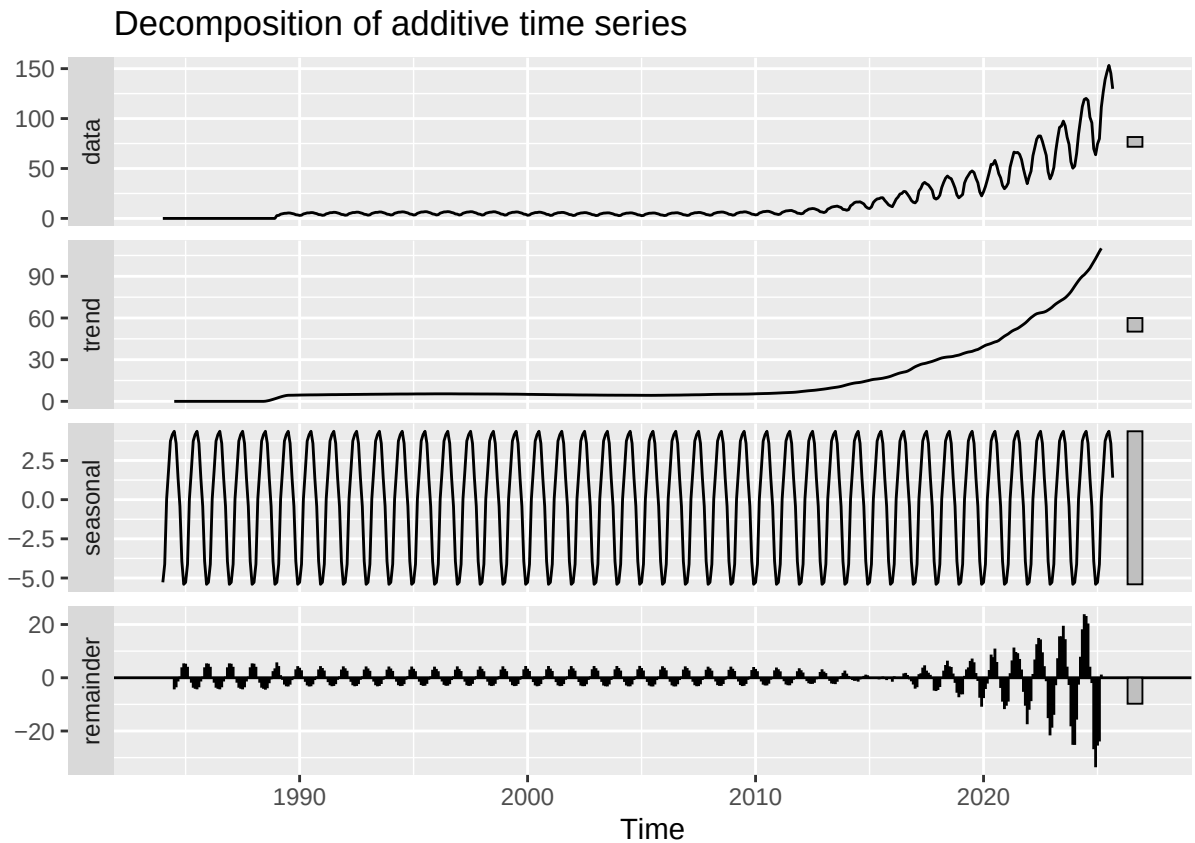
solar_ts <- ts(filtered_df$`Solar Energy Consumption`, frequency=12, start = c(1984,1))

decomposed_wind <- decompose(wind_ts, type = "additive")
autoplot(decomposed_wind)
```



It looks like for my wind time series, most of the series is explained by trend, with a fairly small seasonal component. The remainder does still appear to have a seasonal component.

```
decomposed_solar <- decompose(solar_ts, type = "additive")
autoplot(decomposed_solar)
```

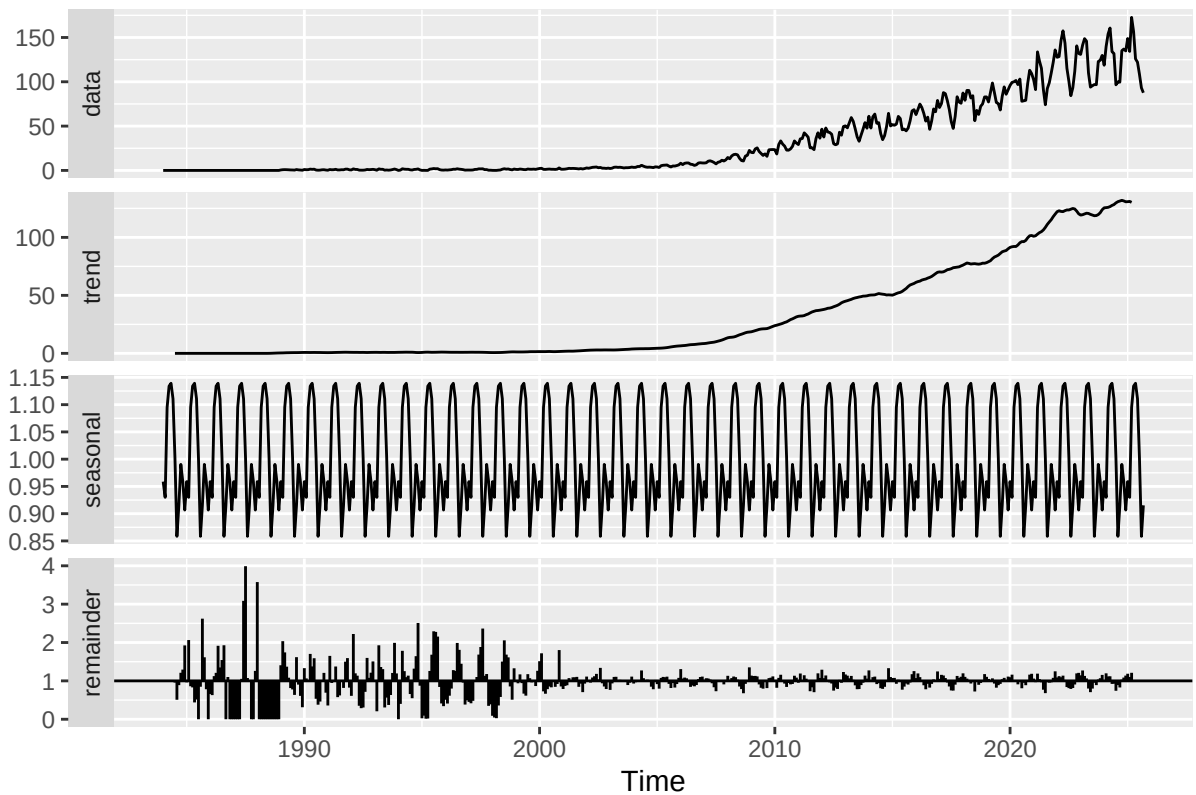
similarly for my solar time series, most is explained by trend, there is some seasonality and there is still seasonality in the remainder. Im not sure what the magnitudes of the decomposed parts indicate.

Q5

Use the `decompose` function again but now change the type of the seasonal component from additive to multiplicative. What happened to the random component this time?

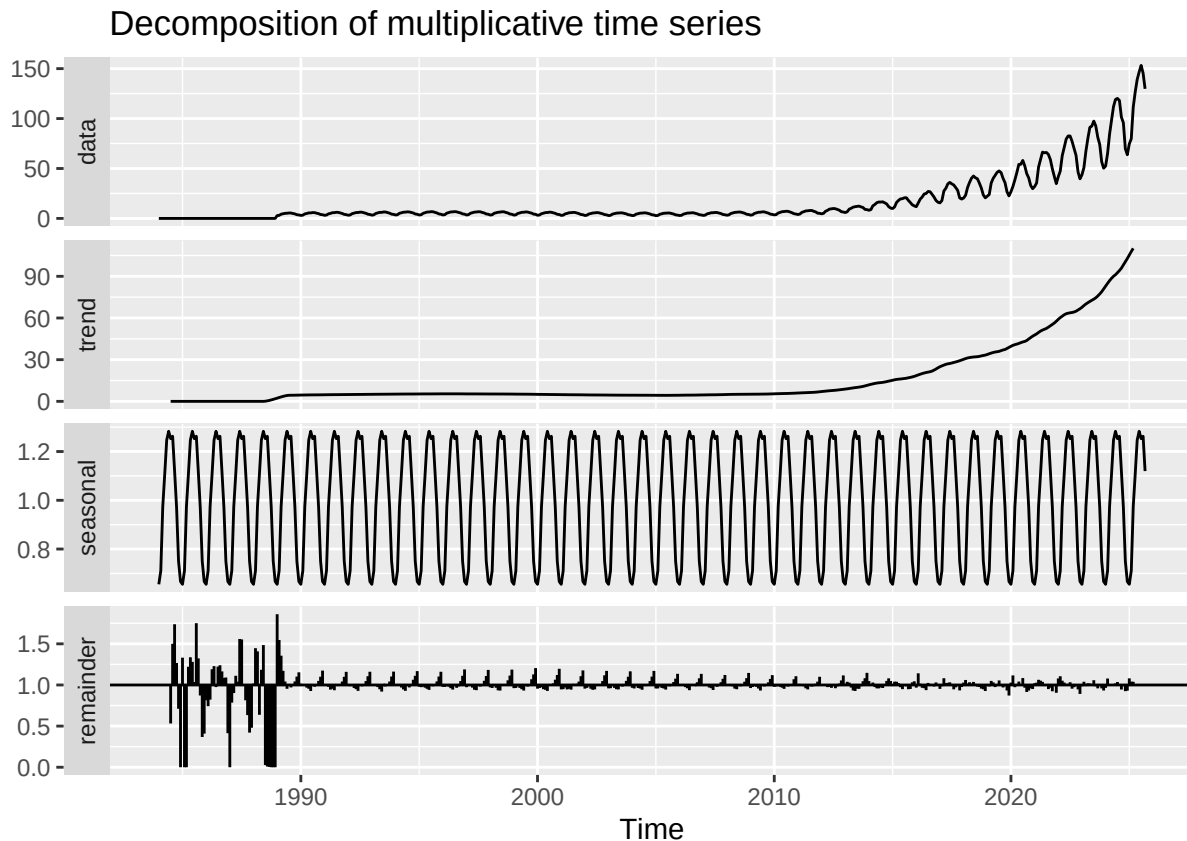
```
decomposed_wind2 <- decompose(wind_ts, type = "multiplicative")
autoplot(decomposed_wind2)
```

Decomposition of multiplicative time series



The random component on my wind data is now much smaller (went from magnitude of 20 to mag of max 4). This decompose looks like it better explains the wind time series

```
decomposed_solar2 <- decompose(solar_ts, type = "multiplicative")
autoplot(decomposed_solar2)
```



Similar results for my solar time series. There is now much less explained by the random component (though there is still something that looks maybe seasonal in there). And almost all is captured by trend and seasonal components. ### Q6

When fitting a model to this data, do you think you need all the historical data? Think about the data from 80s, 90s and early 20s. Are there any information from those years we might need to forecast the next six months of Solar and/or Wind consumption. Explain your response.

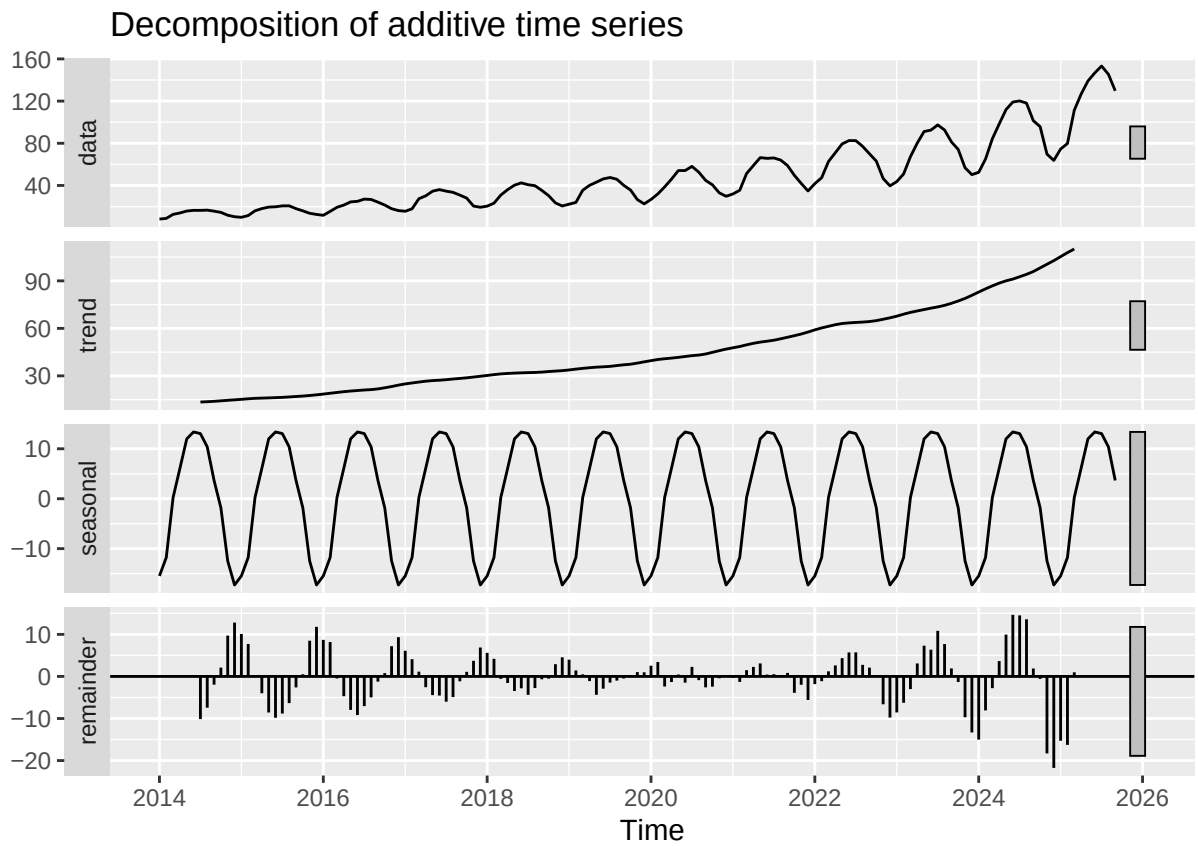
Answer: I don't think the next 6 months of wind data would require having data from the 80s, 90s or early 20s. The time series captures the same seasonal component throughout, and the trend component did not become important (via appearance) until the mid-2000s. Therefore I don't think I would lose anything by not having the first 20 years of data. This is likewise for the solar data and even later (trend isn't significant until after 2010). This is likely due to wind and solar energy infrastructure being deployed at much higher rates in recent history than compared to decades ago - making older data irrelevant for predicting the next 6 months.

Q7

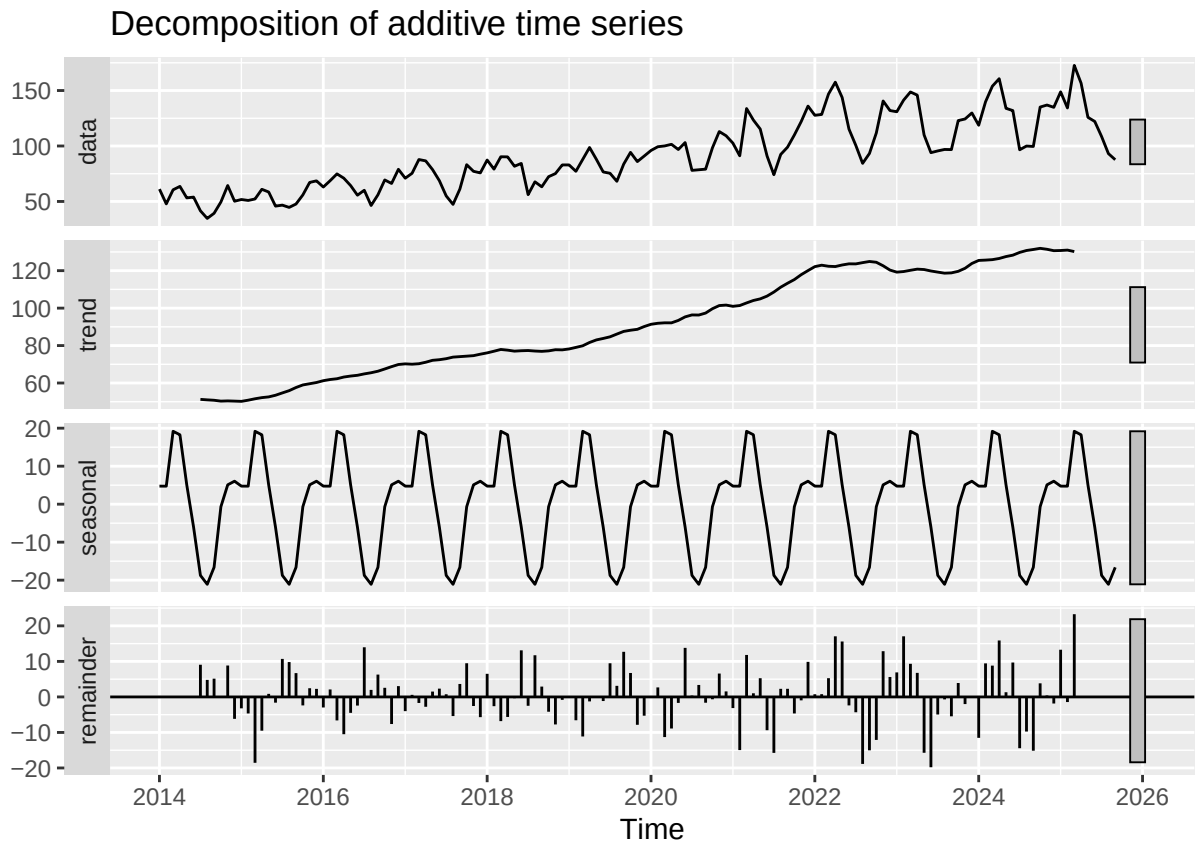
Create a new time series object where historical data starts on January 2014. Hint: use `filter()` function so that you don't need to point to row numbers, i.e. `filter(xxxx, year(Date) >= 2014)`. Apply the `decompose` function `type=additive` to this new time series. Comment on the results. Does the random component look random?

```
recent_ts <- ts(filter(filtered_df, year(Month) >= 2014)[,-1], frequency =12, start = c(2014,1))
```

```
decomposed_recent <- decompose(recent_ts[, 'Solar Energy Consumption'], type = "additive")
autoplot(decomposed_recent)
```



```
decomposed_recent <- decompose(recent_ts[, 'Wind Energy Consumption'], type = "additive")
autoplot(decomposed_recent)
```



Answer: No for neither decomposed plot does the remainder / random component look random. It looks seasonal to me, though the solar plot changes in magnitude over time. The wind random component is slightly less obviously seasonal/ does appear slightly random but still has highs and lows that oscillate and may be seasonal.

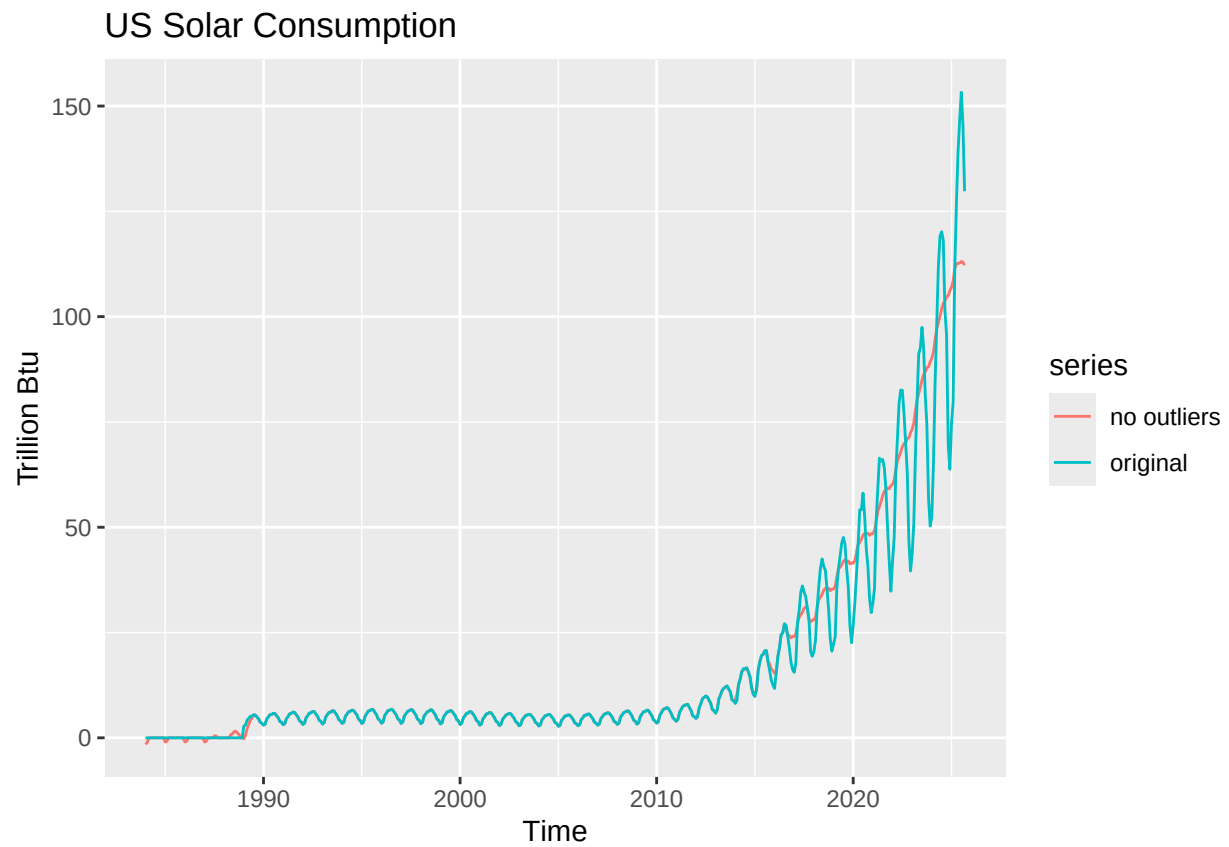
Identify and Remove outliers

Q8

Apply the `tsclean()` to both time series object you created on Q4. Did the function removed any outliers from the series? Hint: Use `autoplot()` to check if there is difference between cleaned series and original series.

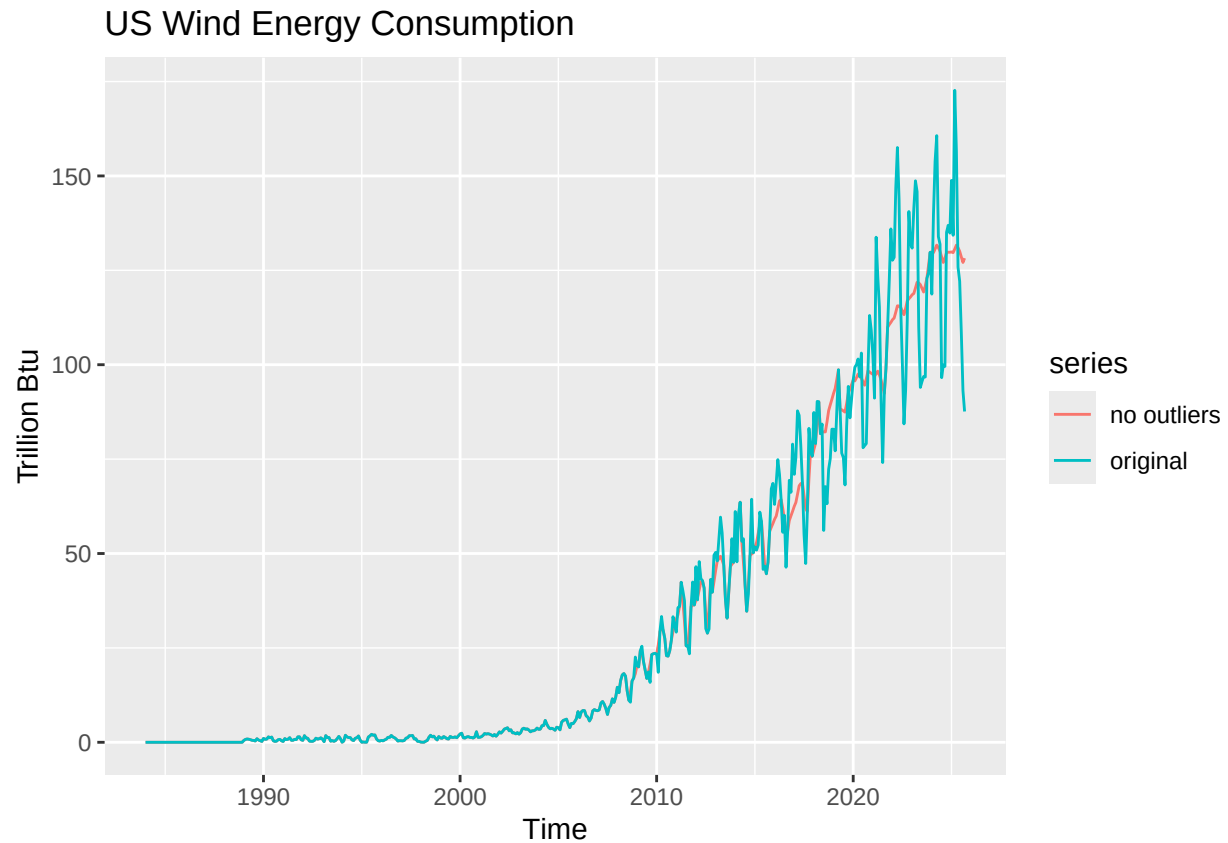
```
clean_solar_ts <- tsclean(solar_ts)

autoplot(clean_solar_ts, series="no outliers")+
  autolayer(solar_ts, series="original")+
  labs(title="US Solar Consumption", y="Trillion Btu")
```



```
clean_wind_ts <- tsclean(wind_ts)

autoplot(clean_wind_ts, series="no outliers")+
  autolayer(wind_ts, series="original")+
  labs(title="US Wind Energy Consumption", y="Trillion Btu")
```

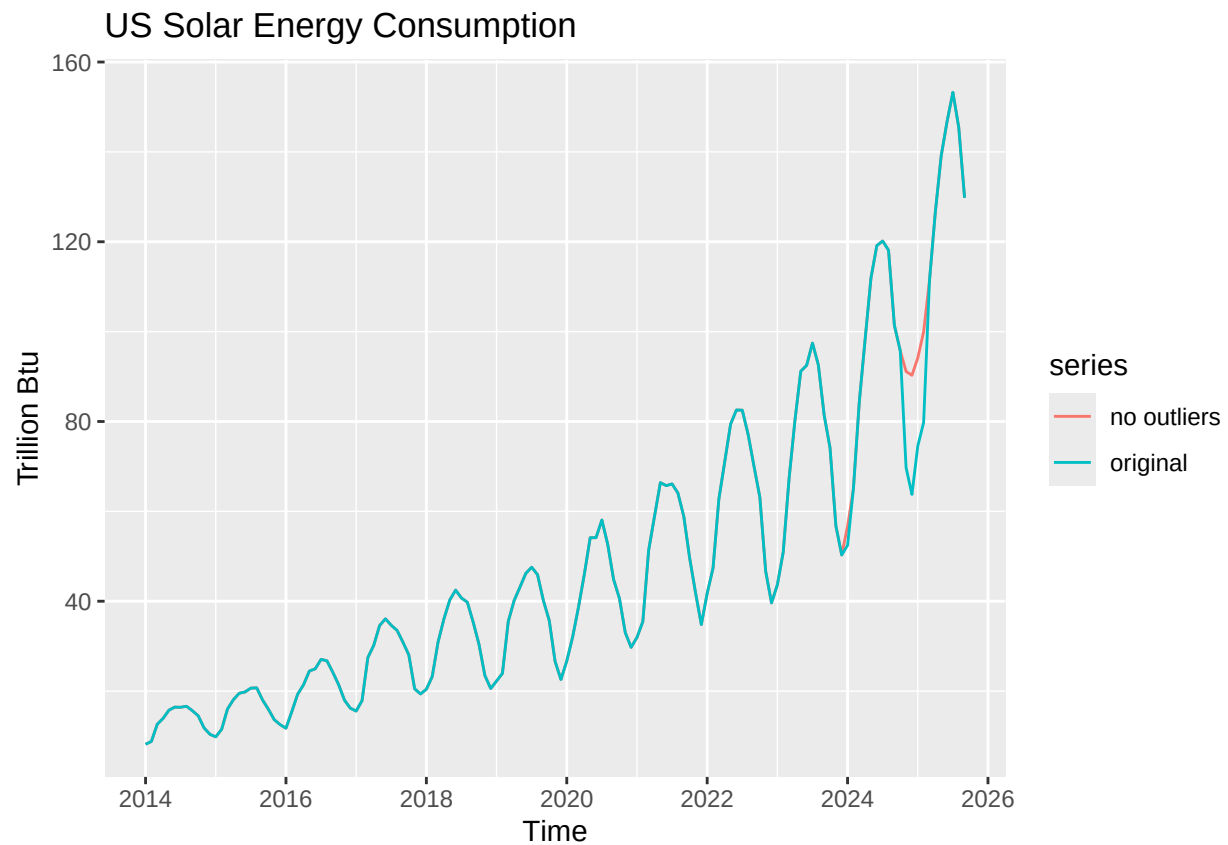


Using autoplot, I can clearly see that the tsclean function removed a ton of outliers, and I'm not sure if I like what it did. I had thought a lot of the fluctuation it ended up removing was seasonality - so I'm not sure I actually want that aspect of my time series removed. It is interesting though. ### Q9

Redo number Q8 but now with the time series you created on Q7, i.e., the series starting in 2014. Using what autoplot() again what happened now? Did the function removed any outliers from the series?

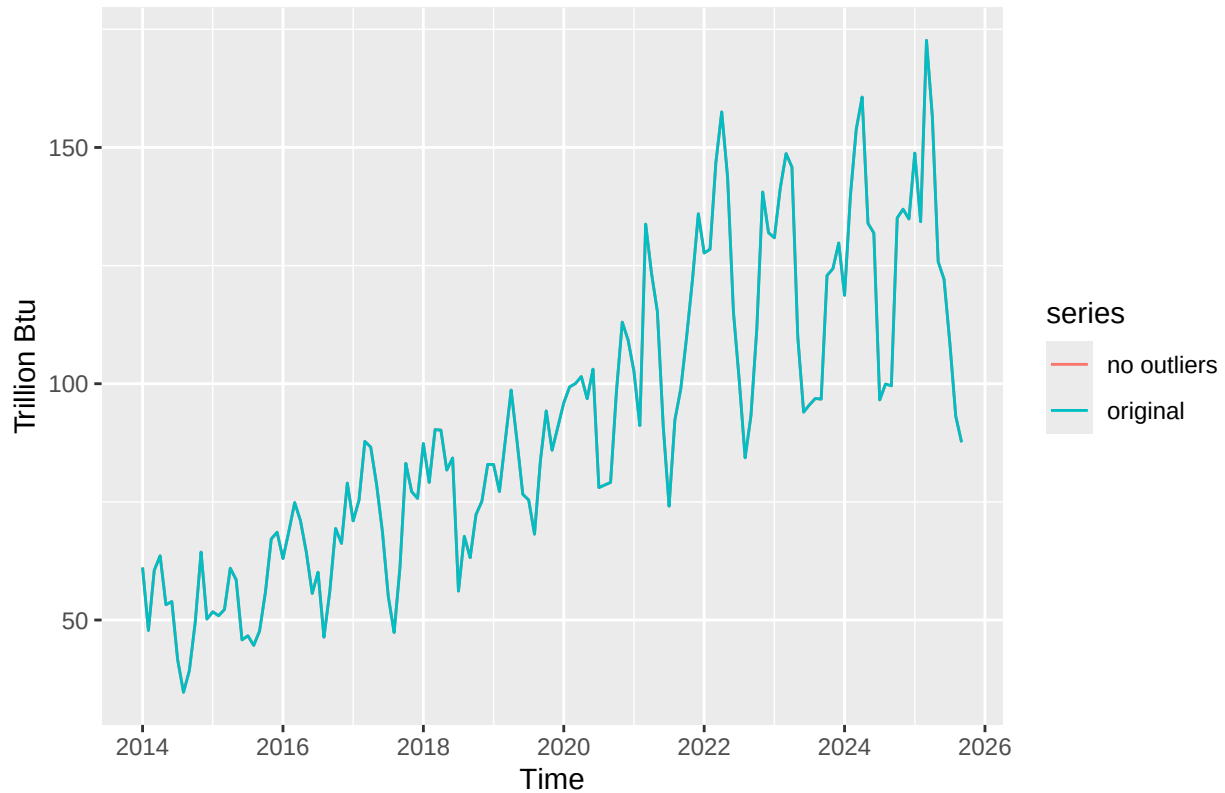
```
clean_solar_ts2 <- tsclean(recent_ts[, 'Solar Energy Consumption'])

autoplot(clean_solar_ts2, series="no outliers")+
  autolayer(recent_ts[, 'Solar Energy Consumption'], series="original")+
  labs(title="US Solar Energy Consumption", y="Trillion Btu")
```



```
clean_wind_ts2 <- tsclean(recent_ts[, 'Wind Energy Consumption'])  
  
autoplot(clean_wind_ts2, series="no outliers")+  
  autolayer(recent_ts[, 'Wind Energy Consumption'], series="original")+  
  labs(title="US Wind Energy Consumption", y="Trillion Btu")
```


US Wind Energy Consumption



Answer: For my solar plot, I only see one instance where an outlier was removed (right around december 2025 where the red line dips lower than the blue original line). For my wind plot, I see no instances where there are two separate lines (the original completely covers the data with no outliers). This shows me that including the older data I thought was irrelevant to predicting the near future was a correct assumption. It would skew what is perceived as an outlier and simplify my data.